

TP Final — LLM Task Manager : un Jira pour LLMs via MCP

Informations

Élément	Valeur
Cours	Architecture Technique Complexe — 5ème année
Type	TP noté
Durée	1 journée (7-8h)
Mode	Binôme
Domaines TOGAF évalués	Business, Application, Data, Technology, Security
Livrable	Repo GitHub + service déployé sur Cloud Run
Notation	/20 (voir barème section 5)

Compétences évaluées

Ce TP final mobilise les compétences acquises au cours des TPs 1 à 4. Vous devez démontrer votre capacité à les **transférer** sur un domaine et un protocole nouveaux.

Compétence	TPs de référence
Concevoir une architecture microservices	TP 1 (Patient Simulator), TP 2 (CGM), TP 4 (Bolus Controller)
Définir un modèle de données et le persister	TP 1 (Patient Simulator), TP 2 (CGM)
Implémenter une API REST avec FastAPI/Pydantic	TP 1 à TP 4
Valider les entrées et sécuriser un service (défense en profondeur)	TP 3 (Pump Service)
Orchestrer plusieurs appels entre services	TP 4 (Bolus Controller)
Déployer sur Cloud Run avec CI/CD	TP 1 à TP 4
Découvrir et intégrer une technologie inconnue (MCP)	Nouveau

1. Le Sujet

Contexte

Les développeurs interagissent de plus en plus avec des LLMs (Claude, ChatGPT) pour écrire du code, déboguer et concevoir l'architecture. Des serveurs MCP comme celui d'Atlassian permettent déjà d'interagir avec Jira depuis une conversation — mais ce sont des **wrappers** autour d'outils complexes conçus pour les humains, avec leurs dizaines de champs, workflows rigides et configurations labyrinthiques.

Et si on concevait un gestionnaire de tâches nativement pensé pour les LLMs ? Plus léger, plus simple, avec des concepts adaptés aux workflows de développement assisté par IA — et construit from scratch sur une architecture cloud moderne.

Le Produit : LLM Task Manager

Vous allez concevoir et construire un **gestionnaire de tâches** accessible à la fois : - via une **API REST** classique (pour les humains et les intégrations) - via le **protocole MCP** (pour que les LLMs puissent créer, lire et modifier des tâches directement depuis une conversation)

Le tout déployé sur **Google Cloud Platform**.

Périmètre fonctionnel

6 entités dans le scope : - **Projet** : créer, lister - **Epic** : créer, lire, modifier, lister, rechercher par mot-clé, filtrer par statut - **Story** : créer, lire, modifier, lister, rechercher par mot-clé, filtrer par statut/priorité/assignée/sprint. Estimation par story points. Workflow de statuts (backlog → todo → in_progress → in_review → done) - **Sprint** : créer, démarrer, clôturer, affecter/retirer des stories. Une story appartient à **un seul sprint actif à la fois** (relation many-to-many historique : une story peut avoir été dans plusieurs sprints successifs, mais jamais dans deux sprints simultanément) - **Commentaire** : ajouter sur une story ou un epic, lister - **Document** : créer (à partir d'un template ou vide), lire, modifier, lister, rechercher. Templates prédéfinis (Problem Statement, Product Vision, Technical Decision Record, Sprint Retrospective)

Toutes ces fonctionnalités sont exposées via **API REST** et via **tools MCP**.

Hors scope (ne pas implémenter) : - Notifications (email, Slack, webhooks) - Analytics et dashboards (vélocité, burndown charts) - Gestion multi-tenant complexe (multi-organisations) - Interface web/frontend - Édition collaborative en temps réel sur les documents

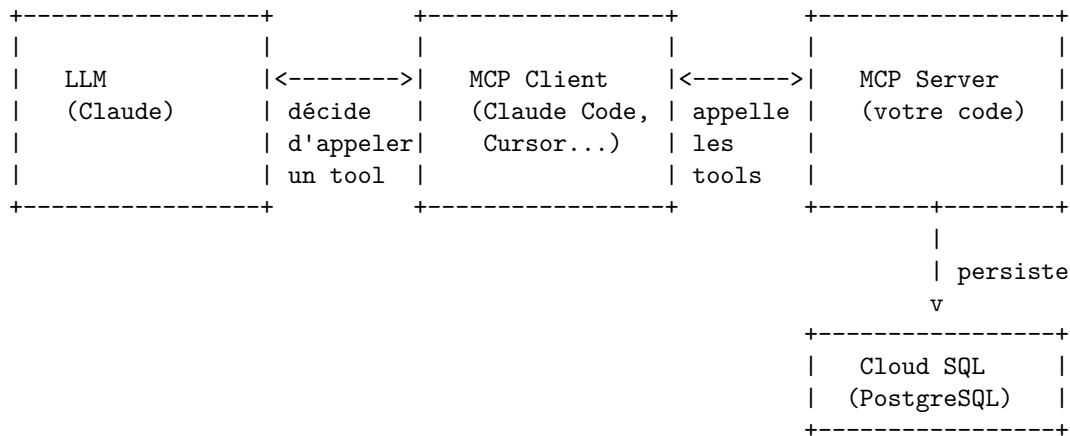
2. Introduction au MCP (Model Context Protocol)

Le MCP est le **seul concept entièrement nouveau** de ce TP. Voici le minimum nécessaire pour démarrer.

Qu'est-ce que MCP ?

Le **Model Context Protocol** est un standard ouvert créé par Anthropic qui permet aux LLMs d'interagir avec des systèmes externes de manière structurée. Concrètement, un **serveur MCP** expose des **tools** (outils) que le LLM peut appeler pendant une conversation. Le LLM ne génère pas de requêtes HTTP lui-même — c'est le **client MCP** (intégré dans Claude Desktop, Claude Code, Cursor, etc.) qui orchestre l'appel.

Architecture



Un tool MCP, c'est quoi ?

Un tool MCP est une **fonction avec un nom, une description et des paramètres typés**. Le LLM lit la description pour décider quand l'appeler. Exemple de signature (pas d'implémentation) :

```
@mcp.tool()
async def create_story(
    project_id: str,
    epic_id: str,
    title: str,
    description: str,
    story_points: int = 0,
    priority: str = "medium"
```

```

) -> dict:
    """Crée une nouvelle story dans un epic.

    Args:
        project_id: Identifiant du projet
        epic_id: Identifiant de l'epic parent
        title: Titre de la story
        description: Description détaillée (critères d'acceptation)
        story_points: Estimation de l'effort (0, 1, 2, 3, 5, 8, 13)
        priority: Priorité (low, medium, high, critical)

    Returns:
        La story créée avec son identifiant unique
    """
    ...

```

Ressources pour démarrer

Ressource	URL
Documentation officielle MCP	https://modelcontextprotocol.io
SDK Python mcp	https://github.com/modelcontextprotocol/python-sdk
Quickstart serveur MCP	https://modelcontextprotocol.io/quickstart/server
Liste de serveurs MCP existants	https://github.com/modelcontextprotocol/servers
Cloud SQL for PostgreSQL (GCP)	https://cloud.google.com/sql/docs/postgres
Connecter Cloud Run à Cloud SQL	https://cloud.google.com/sql/docs/postgres/connect-cloud-run
SQLModel (SQLAlchemy + Pydantic)	https://sqlmodel.tiangolo.com

Important : explorez les exemples de serveurs MCP existants pour comprendre la structure, mais votre implémentation doit être la vôtre.

3. Phase 1 — Conception architecturale

Avant d'écrire la moindre ligne de code, vous devez concevoir l'architecture de votre système en passant par les **5 domaines TOGAF**. C'est cette démarche d'architecte que l'on évalue prioritairement.

Livrable : un document `ARCHITECTURE.md` dans votre repo GitHub, contenant les 5 sections ci-dessous.

3.1 Business Architecture

Objectif : Définir **pour qui** et **pourquoi** vous construisez ce système.

Livrables attendus : - **2 personas minimum** avec leur contexte, frustrations et besoins - **1 JTBD principal** au format : *“En tant que [persona], je veux [action] afin de [bénéfice]”* - **3 cas d'usage concrets** décrivant comment chaque persona interagit avec le système - **Règles métier du domaine** : définir au moins **3 règles métier** qui encadrent le fonctionnement du système (exemples : story points uniquement dans la suite de Fibonacci, impossible de clôturer un sprint avec des stories en cours, nombre max de stories dans un sprint, une story ne peut pas sauter d'étape dans le workflow, etc.)

Question à vous poser : qui sont les utilisateurs d'un Jira pour LLMs ? Pensez au-delà du développeur.

3.2 Application Architecture

Objectif : Définir la **décomposition en services**, les **interfaces** et les **patterns de communication**.

Livrables attendus : - **Diagramme d'architecture** montrant les services, leurs interactions et les protocoles - **Liste des endpoints REST** avec méthode HTTP, path, paramètres et réponse attendue - **Liste des tools MCP** avec nom, description, paramètres et retour - **Justification architecturale** : avez-vous choisi 1 service unique (REST + MCP dans le même process) ou 2 services séparés ? Pourquoi ? - **Contrat de validation** : pour chaque entité, quelles contraintes sur les champs ? (longueurs max, formats, valeurs autorisées pour les statuts/priorités, codes HTTP en cas d'erreur de validation, etc.)

Rappel : les patterns d'architecture vus en cours (stateless, orchestration, separation of concerns) s'appliquent ici aussi. Pensez au nombre de tools MCP nécessaires pour couvrir les 6 entités (projet, epic, story, sprint, commentaire, document).

3.3 Data Architecture

Objectif : Définir le **modèle de données relationnel** et la **stratégie de persistance**.

Livrables attendus : - **Schéma des tables PostgreSQL** : quelles tables, quelles colonnes, quels types, quelles contraintes (NOT NULL, UNIQUE, CHECK, etc.) - **Relations entre entités** : clés étrangères, cardinalités (1-N, N-N). Comment liez-vous projets, epics, stories, sprints, commentaires et documents ? Comment gérez-vous la relation many-to-many historique entre stories et sprints (table de jonction) ? Rappel : une story appartient à un seul sprint actif à la fois, mais peut avoir transité par plusieurs sprints au fil du temps. - **Diagramme entité-relation** (ERD) montrant les tables et leurs liens - **Index nécessaires** : quels index pour optimiser vos requêtes de filtrage (stories par statut, par sprint, par assignée, recherche textuelle, etc.) ? - **Gestion des templates de documents** : où et comment stockez-vous les structures prédéfinies ?

Rappel : un modèle relationnel bien conçu évite la duplication de données et garantit l'intégrité référentielle. Pensez aux contraintes SQL comme un premier niveau de validation (avant même Pydantic).

3.4 Technology Architecture

Objectif : Définir l'**infrastructure** et les **choix techniques**.

Livrables attendus : - **Diagramme de déploiement GCP** montrant les services, régions et flux réseau (notamment la connexion Cloud Run → Cloud SQL) - **Tableau des services GCP utilisés** avec justification de chaque choix - **Configuration de scaling** : min/max instances, cold starts, stratégie - **Stratégie de connexion BDD** : comment Cloud Run se connecte à Cloud SQL (Cloud SQL Auth Proxy, connecteur Python, Unix socket, etc.) - **Pipeline CI/CD** : étapes du pipeline Cloud Build (install, migrations, tests, deploy)

3.5 Security Architecture

Objectif : Définir comment le système est **protégé**.

Livrables attendus : - **Stratégie d'authentification REST** : méthode choisie (API key, JWT, autre) et justification - **Stratégie de sécurité MCP** : transport choisi (stdio, SSE, streamable HTTP), politique d'exposition des tools, contrôle d'accès - **Matrice de permissions** : rôles, droits par entité, cohérence entre REST et MCP - **Mesures de protection** : injections SQL, abus (rate limiting), accès non autorisé - **Stratégie de défense en profondeur** : couches de validation identifiées (Pydantic, SQL, logique métier), partage entre les deux points d'entrée

4. Phase 2 — Implémentation et déploiement

Objectif : Traduire votre architecture en **code fonctionnel déployé**, en démontrant votre maîtrise de la stack technique et votre capacité à livrer un service opérationnel.

Livrables attendus :

- **Repo GitHub** contenant : le code source, un README (instructions de setup et d'exécution), le document ARCHITECTURE.md, les tests
- **Tests automatisés passants** : unitaires (logique métier, modèles Pydantic) et au moins 1 test d'intégration
- **Service déployé sur Cloud Run** : accessible via son URL, endpoint `/health` fonctionnel
- **Démonstration de 20 min** : walkthrough architecture (5 min), démo REST (5 min), démo MCP (5 min), questions de l'enseignant (5 min)

5. Barème et checklist de rendu

Chaque item est à la fois un livrable attendu et un critère de notation. Cochez au fur et à mesure pour vous assurer de ne rien oublier.

Vue d'ensemble

Catégorie	Points	Poids
Architecture	8	40%
Implémentation	8	40%
Déploiement et démo	4	20%
Total	20	100%

Architecture (8 pts)

Repository GitHub : repo accessible par l'enseignant, README.md avec instructions de setup et d'exécution, ARCHITECTURE.md avec les 5 domaines TOGAF.

Business Architecture (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Absent ou personas génériques sans lien avec le produit. Pas de règles métier.
Partiel	0.5	Personas présents mais superficiels. JTBD vague. Règles métier triviales ou incomplètes.
Satisfaisant	1	2 personas avec contexte clair. JTBD et cas d'usage cohérents. 3 règles métier pertinentes.
Excellent	1.5	Personas différenciés (LLM vs humain). Cas d'usage démontrant les deux points d'entrée. Règles métier reflétant une vraie compréhension du domaine (ex. contraintes de workflow, invariants métier).

Application Architecture (2 pts)

Niveau	Points	Descripteur
Insuffisant	0	Pas de diagramme ou diagramme incohérent. Endpoints listés sans détail.

Niveau	Points	Descripteur
Partiel	0.5-1	Diagramme présent mais incomplet. Endpoints REST listés sans contrat de validation. Tools MCP absents ou partiels. Pas de justification architecturale.
Satisfaisant	1.5	Diagramme cohérent. Endpoints et tools MCP couvrant les 6 entités. Contrats de validation présents. Choix 1 vs 2 services justifié.
Excellent	2	Architecture bien décomposée avec séparation claire des responsabilités. Contrats de validation précis (types, longueurs, codes d'erreur). Descriptions MCP optimisées pour la compréhension par un LLM. Justification architecturale argumentée avec trade-offs.

Data Architecture (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Pas de schéma SQL ou schéma non normalisé (données dupliquées, pas de clés étrangères).
Partiel	0.5	Tables principales présentes mais relations incomplètes. Pas de table de jonction. Index absents.
Satisfaisant	1	Schéma normalisé avec FK, table de jonction stories<->sprints. ERD lisible. Index sur les colonnes de filtrage principales.
Excellent	1.5	3NF respectée. Contraintes CHECK pertinentes (ex. story_points IN Fibonacci). Index couvrant les requêtes de recherche et filtrage. Stratégie de templates documentée (table dédiée ou seed data).

Technology Architecture (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Pas de diagramme de déploiement. Services GCP listés sans justification.
Partiel	0.5	Diagramme basique. Connexion Cloud Run → Cloud SQL non détaillée. Pas de pipeline CI/CD.
Satisfaisant	1	Diagramme avec services, région et flux réseau. Stratégie de connexion BDD claire (Auth Proxy ou connecteur Python). Pipeline CI/CD avec étapes principales.
Excellent	1.5	Configuration de scaling documentée (min/max instances). Stratégie de cold start. Pipeline CI/CD détaillé (install → migrations → tests → deploy). Justification de chaque service GCP.

Security Architecture (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Sécurité non abordée ou mention vague (“on sécurisera”).
Partiel	0.5	Authentification REST définie mais sécurité MCP ignorée. Pas de matrice de permissions.
Satisfaisant	1	Auth REST + sécurité MCP définies. Permissions cohérentes entre les deux points d’entrée. Protections de base mentionnées (SQL injection, rate limiting).
Excellent	1.5	Stratégie complète avec défense en profondeur documentée (couches Pydantic, logique métier, SQL). Transport MCP justifié. Matrice de permissions détaillée par rôle et par entité.

Implémentation (8 pts)

API REST fonctionnelle (2 pts)

Niveau	Points	Descripteur
Insuffisant	0	API non fonctionnelle ou seulement 1-2 entités implémentées.
Partiel	0.5-1	CRUD sur 3-4 entités. Pas de filtrage ni de recherche. Codes HTTP incohérents.
Satisfaisant	1.5	CRUD sur les 6 entités. Filtrage et recherche fonctionnels. Codes HTTP corrects. Templates présents.
Excellent	2	Couverture complète avec workflow de statuts validé (transitions interdites rejetées). Templates fonctionnels. Réponses paginées et structurées.

Serveur MCP fonctionnel (2 pts)

Niveau	Points	Descripteur
Insuffisant	0	Serveur MCP absent ou non fonctionnel.
Partiel	0.5-1	Quelques tools fonctionnels mais couverture partielle (ex. seulement création, pas de recherche). Descriptions vagues.
Satisfaisant	1.5	Tools couvrant les opérations principales des 6 entités. Descriptions et paramètres typés.
Excellent	2	Parité complète avec l’API REST. Descriptions optimisées pour un LLM (claires, contextuelles). Gestion d’erreurs avec messages explicites.

Persistence Cloud SQL (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Pas de connexion Cloud SQL ou tables non créées.

Niveau	Points	Descripteur
Partiel	0.5	Tables créées mais schéma divergent de ARCHITECTURE.md. Pas de FK ou contraintes manquantes.
Satisfaisant	1	Tables conformes au schéma documenté. FK et contraintes de base respectées. Connexion Cloud Run fonctionnelle.
Excellent	1.5	Schéma parfaitement cohérent avec ARCHITECTURE.md. Intégrité référentielle complète. Migrations gérées proprement.

Tests (1.5 pts)

Niveau	Points	Descripteur
Insuffisant	0	Aucun test ou tests qui ne passent pas.
Partiel	0.5	Quelques tests unitaires mais pas de test d'intégration. Assertions triviales (ex. assert True).
Satisfaisant	1	Tests unitaires sur les modèles Pydantic et la logique métier. Au moins 1 test d'intégration (cycle CRUD). Assertions vérifiantes.
Excellent	1.5	Tests couvrant les cas nominaux et les cas d'erreur (validation rejetée, transitions interdites). Test d'intégration complet.

Qualité du code (1 pt)

Niveau	Points	Descripteur
Insuffisant	0	Code monolithique dans un seul fichier. Pas de modèles Pydantic.
Partiel	0.5	Séparation basique (routes vs modèles). Pydantic utilisé mais validation incomplète.
Satisfaisant	0.75	Structure claire (routes, modèles, services). Pydantic pour toutes les entrées. Nommage cohérent.
Excellent	1	Séparation exemplaire des responsabilités. Validation multi-couches (Pydantic + logique + SQL). Code lisible et idiomatique.

Déploiement et démo (4 pts)

Cloud Run opérationnel (2 pts)

Niveau	Points	Descripteur
Insuffisant	0	Service non déployé ou URL non fonctionnelle.
Partiel	1	Service déployé mais instable (erreurs fréquentes, cold starts excessifs) ou /health non implémenté.
Satisfaisant / Excellent	2	Service déployé en europe-west1, URL fonctionnelle, /health répond HTTP 200.

Démonstration live (2 pts)

Niveau	Points	Descripteur
Insuffisant	0	Pas de démo ou démo non fonctionnelle.
Partiel	0.5-1	Démo REST fonctionnelle mais pas de démo MCP. Réponses vagues aux questions.
Satisfaisant	1.5	Démo REST et MCP fonctionnelles. Réponses correctes aux questions architecturales.
Excellent	2	Démo fluide et bien préparée sur les deux canaux. Réponses démontrant une compréhension approfondie des choix et trade-offs.

Bonus (hors barème, jusqu'à +2 pts)

Bonus	Points
Tests E2E sur le service déployé	+0.5
Pipeline Cloud Build fonctionnel et automatisé	+0.5
Gestion d'erreurs avancée (retry, messages explicites)	+0.5
Documentation API (Swagger/OpenAPI personnalisé)	+0.5

6. Contraintes et règles

Contraintes techniques

Couche	Technologie
Langage	Python 3.11+
Package manager	uv
Framework API	FastAPI
Validation	Pydantic v2
ORM	SQLAlchemy ou SQLModel
Protocole LLM	SDK officiel Python MCP (<code>pip install mcp</code>)
Base de données	Cloud SQL PostgreSQL
Déploiement	Cloud Run, europe-west1
Version control	GitHub

Cette stack est **non négociable**. Tous vos choix d'architecture doivent s'inscrire dans ce cadre.

Bonne chance. Montrez que vous savez penser en architecte.